

Breaking Down the Fundamentals of GIT

Lania Laura Quadros
School of Mechanical
Manipal Institute of Technology
Manipal, India
quadroslania2@gmail.com

Abstract— Git is a distributed version control system used to manage changes in software projects and maintain their complete development history. This report presents the fundamental concepts and practical operations of Git, including the working directory, staging area, local and remote repositories, commits and Git objects, branching, HEAD, merging and merge conflict resolution. It also discusses pushing, force pushing, pulling, fetching, rebasing, logging, reference logs, and stashing. The study combines conceptual understanding with practical experimentation using Git commands and the Learn Git Branching platform, covering both local and remote repository operations. The objective is to understand how Git manages project history, enables parallel development, and supports collaboration through distributed version control.

Keywords—Git, version control, repository, commits, branching, merging, rebasing, GitHub, staging area, remote repository

1. FUNDAMENTALS OF GIT

A. Version Control System

A version control system, or VCS, tracks the history of changes as people and teams collaborate on projects together. VCSs give each contributor a unified and consistent view of a project, surfacing work that's already in progress. Seeing a transparent history of changes, who made them, and how they contribute to the development of a project helps team members stay aligned while working independently.

B. Git and its Workflow

Git is the most popular distributed version control system. Git lets developers see the entire timeline of their changes, decisions, and progressions of any project in one place. With a DVCS like Git, collaboration can happen any time while maintaining source code integrity. Using branches, developers can safely propose changes to production code.

The general workflow is:

1) *Working Directory*: The actual folder contains the project files on the user's computer. It is the area where the files are created, modified, or deleted during normal development.

The current state of the working directory can be checked using 'git status'.

2) *Staging area*: Also called the index, is an intermediate area between the working directory and the repository. It allows the user to select exactly which changes should be included in the next commit.

Changes are added to the staging area using 'git add <filename>' and multiple files can be staged by using 'git add .'.

3) *Repository*: It contains Git history and objects stored on the user computer. It is located inside the hidden .git directory created when 'git init' is executed.

C. About repositories

A repository, or Git project, encompasses the entire collection of files and folders associated with a project, along with each file's version history. The file history appears as snapshots in time called commits. The commits can be organized into multiple lines of development called branches. Because Git is a DVCS, repositories are self-contained units and anyone who has a copy of the repository can access the entire codebase and its history. Using the command line or other ease-of-use interfaces, a Git repository also allows for: interaction with the history, cloning the repository, creating branches, committing, merging, comparing changes across versions of code, and more.

1) *Local repository*: The git repository stored on your own computer. It contains your project files, commits, branches, and Git history. You can work on commit changes without internet.

2) *Remote repository*: A Git repository stored on another computer/server. Usually on platforms like GitHub. It is used to store, share, and collaborate on the project.

D. How GitHub works

Through platforms like GitHub, Git also provides more opportunities for project transparency and collaboration. GitHub hosts Git repositories and provides developers with tools to ship better code through command line features, issues (threaded discussions), pull requests, code review, or the use of a collection of free and for-purchase apps in the GitHub Marketplace.

Then, using the GitHub flow, developers simply create a branch to work on updates, commit changes to save them, open a pull request to propose and discuss changes, and merge pull request once everyone is on the same page.

II. WHAT A COMMIT ACTUALLY IS

A Git commit is a permanent snapshot of your entire project's files and folders at a specific point in time.

A. What does a commit contain?

- 1) *A unique ID*: A long code that identifies the commit.
- 2) *A pointer to a tree object*: The root directory structure of your project.
- 3) *Metadata*: The author's name and email, timestamp, and our commit message.
- 4) *Parent commit(s)*: A link to the previous commit(s) that came before it, forming your project's history chain.

B. How commits are stored

Git stores everything inside the hidden `.git` folder in your project directory as compressed objects. Git uses three main types of objects to make the storage fast and efficient:

- 1) *Blobs*: Short for ‘Binary large object,’ a blob stores the raw contents of a file, but not the file name.
- 2) *Trees*: A tree object represents a folder. It maps file names and folder names to their matching blobs or sub-trees.
- 3) *Commits*: The commit object points to the root tree object of the project and holds the metadata and parent links.

III. BRANCHING

Branching is one of the most fundamental features of Git that allows different lines of development to exist within the same repository. When a new commit is created, the branch reference moves forward to the point to that new commit.

The 'git branch' command is used to create and view branches, while 'git switch' can be used to move between branches. When switching branches, Git changes the working directory to match the commit referenced by the selected branch.

```
lania@Lappy MINGW64 ~/OneDrive/Documents/git-demo (master)
$ git switch feature
```

A. Main/Master Branch

Commonly used as the primary branch of the repository. Additional branches can be created for developing features, experimenting with changes, or fixing bugs without directly modifying the main development line.

```
lania@Lappy MINGW64 ~/OneDrive/Documents/git-demo (master)
$ git branch
* master
```

B. HEAD

HEAD is special reference that indicates the commit currently being worked on. Normally, HEAD points to a branch, and that branch points to the latest commit. When a new commit is created, the branch and consequently HEAD moves to the new commit.

A detached HEAD occurs when HEAD points directly to a commit instead of a branch. Changes can still be made and committed in this stage, but the commits may become difficult to locate unless a branch or another reference is created to preserve them.

IV. MERGING

The process of combining the changes and histories of two branches. It is commonly used when development on a feature branch is complete and its changes need to be incorporated into the main branch.

```
lania@Lappy MINGW64 ~/OneDrive/Documents/git-demo (main)
$ git log --oneline --graph --all
* 8f6f3bd (HEAD -> main, merge-demo) Add merge change
* 451e345 (master) Initial commit
```

A. Three-Way Merge

A three-way merge is required when two branches have diverged and both contain commits that are not present in either branch. Git uses three points to perform the merge:

- 1) The current tip of the target.
- 2) The tip of the branch being merged.
- 3) Their common ancestor.

Git combines the changes made on both branches and may create a new merge commit containing both histories.

B. Merge Conflicts

A three-way merge is required when two branches have diverged and both contain commits that are not present in either branch. Git uses three points to perform the merge:

A merge conflict occurs when Git cannot automatically determine how to combine changes from two branches. This commonly happens when the same lines of a file have been modified differently on both branches.

Git marks the conflicting section using conflict markers such as ‘<<<<<<’, ‘=====’, and ‘>>>>>>’. The developer must inspect the conflicting code, decide which changes should remain, remove the conflict markers, and save the corrected file. Conflicts are therefore not errors in Git itself, but situations where human input is required to determine the correct final version.

V. PUSHING

The git command ‘git push’ transfers commits from the local repository to the remote repository. For example. Pushing the main branch sends local commits that are missing from the remote and updates the remote branch when the operation is permitted.

```
lania@Lappy MINGW64 ~/OneDrive/Documents/git-demo (conflict-a)
$ git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), 490 bytes | 490.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/UglyAlpaca/Demo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Git only allows to push when the remote branch can be updated without discarding existing remote commits. A force push overrides these restrictions and allows the remote branch reference to be changes to match the local history.

```
lania@Lappy MINGW64 ~/OneDrive/Documents/git-demo (main)
$ git push --force-with-lease
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/UglyAlpaca/Demo.git
+ 8f6f3bd...451e345 main -> main (forced update)
```

VI. PULLING AND FETCHING

A. Git fetch

Downloads new commits, branches and other references from the remote repository without changing the current working branch. This allows the user to inspect remote changes before deciding whether to integrate them.

B. Git pull

Combines fetching with an integration operation. After fetching the remote changes, Git normally integrates then using either a merge or a rebase, depending on the command and repository configuration.

The main difference is therefore that fetching only obtains the remote information, while pulling obtains the information and integrates into the current branch.

VII. REBASING

Rebasing is an alternative method of integrating changes between branches. Instead of creating a merge commit, Git moves or replies a series of commits onto a new base commit.

Because the commits are recreated, their commit hashes change. This means that rebasing effectively rewrites history and it is useful for maintaining a clean, linear history.

Interactive rebase provides greater control over sequence of commits. Using ‘git rebase -i’, commits can be reordered, combined, edited, or removed.

VIII. LOGS AND REFLOGS

A. Git log

Git maintains a record of commits that can be inspected using log commands. The ‘git log’ command displays information such as commit hashes, authors, dates, and commit messages.

```
lania@lappy MINGW64 ~/OneDrive/Documents/git-demo (rebase-demo)
$ git log
commit 9248c331be7c736f48af2a9c1cdf0ca694f73e85 (HEAD -> rebase-demo)
Author: lania Quadros <quadros.lania2@gmail.com>
Date: Sat Sep 5 05:34:11 2026 +0530

    Add rebase change

commit 451e34564dd5ccfd21ca4406a6fe211307e6bd9d (origin/main, origin/HEAD, master, main)
Author: lania Quadros <quadros.lania2@gmail.com>
Date: Sat Sep 5 03:34:44 2026 +0530

    Initial commit
```

‘git log –oneline’ provides a compact representation of the history, making it easier to understand the sequence of commits.

```
lania@lappy MINGW64 ~/OneDrive/Documents/git-demo (rebase-demo)
$ git log --oneline --graph --all
* 9248c33 (HEAD -> rebase-demo) Add rebase change
* 77002d9 (conflict-a) Change from main
* 8efe09e Change from branch A
* 8f6f3bd (merge-demo) Add merge change
/
* 451e345 (origin/main, origin/HEAD, master, main) Initial commit
```

B. Git reflog

The reference log records changes to references such as HEAD and local branches. Unlike the normal commit log, which mainly follows the ancestry of commits, it records movements of references over time.

‘git reflog’ is therefore particularly useful for recovering work after operations such as reset, rebase, or accidental branch movement.

IX. STASHING

‘git stash’ provides a temporary method of storing uncommitted changes. It is useful when the working directory contains unfinished changes but the developer needs to temporarily switch branches or perform another operation.

The ‘git stash’ command saves the current modifications and returns the working directory to a clean state. The saved

changes can later be restored using ‘git stash apply’ or ‘git stash pop’.

Unlike a normal commit, a stash is intended for temporary storage rather than becoming part of the normal project history.

```
lania@lappy MINGW64 ~/OneDrive/Documents/git-demo (rebase-demo)
$ git stash
No local changes to save

lania@lappy MINGW64 ~/OneDrive/Documents/git-demo (rebase-demo)
$ git status
On branch rebase-demo
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    stash.txt

nothing added to commit but untracked files present (use "git add" to track)

lania@lappy MINGW64 ~/OneDrive/Documents/git-demo (rebase-demo)
$ git stash pop
No stash entries found.

lania@lappy MINGW64 ~/OneDrive/Documents/git-demo (rebase-demo)
$ git status
On branch rebase-demo
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    stash.txt

nothing added to commit but untracked files present (use "git add" to track)
```

X. CONCLUSIONS

This task provided practical and conceptual understanding of Git as a distributed version control system. The work covered commits and the storage models, branches, HEAD, logs, reflogs, merging, rebasing, conflict resolution, stashing, pushing, pulling, fetching and force pushing.

Interactive practice through Learn Git branching complemented hands-on work in a local repository. The final practical repository demonstrated that required local and remote operations and contained the task 1 solutions.

ACKNOWLEDGMENTS

I extend my deepest gratitude to the seniors in the Driverless domain of the student project, Formula Manipal, whose resource and assistance have been invaluable, I would like also like to acknowledge my peers for their camaraderie and the stimulating discussions that inspired me throughout my task 2 journey

REFERENCES

- [1] GitHub Docs, About Git “<https://docs.github.com/en/get-started/using-git/about-git>”
- [2] Learn Git Branching, Main and Remote Tabs “<https://learngitbranching.js.org/>”
- [3] Git-config(1) Manual Page, Git-config – Get and set repository or global options, “<file:///C:/Program%20Files/Git/mingw64/share/doc/git-doc/git-config.html>”
- [4] Git Tutorial For Dummies, “<https://www.youtube.com/watch?v=mJ-qvsxPHpY>”
- [5] Git Tutorial for Beginners: Command-Line Fundamentals, “<https://www.youtube.com/watch?v=HVsySz-h9r4>”